Milestone 3 Report – SmartLink Software Design Project: SmartLink – AI Features & Advanced Functionality

1. Team Members

Name	ID
Omar Khaled Hussein	22010962
Ayman Ibrahim Mohamed Kotb	22010656
Ali El-Deen Maher Abd El-Moniem	22010934
Ali Mostafa Mohamed	22010939
Yaseen Asaad Ahmed Farid	22011349
Farouk Ashraf Farouk	22011012

---

2. Milestone Objective

Objectives: Integrate intelligent and data-driven modules that enhance personalization, recruitment, and system scalability.

---

3. Scope & Implemented Features

Core Scope:

The primary scope of this milestone was to transform the platform from a foundational job portal into an intelligent, data-driven career ecosystem. The focus was on implementing advanced AI and analytics modules to automate key processes, provide deep personalization, and empower administrators with actionable insights.

Implemented Features:

1. **AI-Powered Candidate & Job Matching: A dual-recommendation engine that intelligently surfaces relevant job opportunities to candidates and ranks qualified applicants for employers, moving beyond simple keyword matching.**
2. **Automated Resume & Skill Parser: An intelligent parsing system that extracts structured data (skills, experience, education) from uploaded resumes (PDF), auto-populates candidate profiles, and enriches the platform's skill taxonomy.**
3. **Centralized Analytics Dashboard: A comprehensive admin dashboard providing system-wide KPIs, user engagement metrics, job market trends, and audit logs for data-driven decision-making and platform governance.**
4. **Enhanced Admin Controls: Advanced administrative tools for user management, content moderation, system health monitoring, and configuration of the AI recommendation parameters.**
5. **Basic Direct Messaging (Optional): A foundational real-time messaging interface enabling direct communication between candidates and recruiters within the platform.**
6. **Message Management (Optional): Moderation controls allowing users to delete conversations and report/block other users, with admin oversight for reported content.**
7. **A WebSocket-based notification system that delivers instant updates to users for:**
   - **Connection requests (received, accepted)**
   - **Job applications and status updates**
   - **Company follows and engagement**
   - **Post interactions (likes, comments)**

---

4. Functional Requirements

FR5.2: AI-Based Job Recommendations

- 5.2.1: The system shall analyze a candidate's profile (skills, experience) and to generate a personalized feed of recommended job listings.
- 5.2.2: The system shall provide employers with an AI-ranked list of candidate recommendations for each job posting, based on profile match, skill relevance, and experience fit.

FR5.3: Resume Analysis System

- 5.3.1: The system shall allow candidates to upload resumes in PDF and DOCX formats.
- 5.3.2: Upon upload, the system shall automatically parse the resume to extract key fields: Skills, Work Experience (job titles, companies, durations), Education, and Contact Information.
- 5.3.3: The parsed data shall be used to pre-fill the candidate's profile, which they can then review, edit, and save.

FR4.3: System Analytics and Admin Enhancements

- 4.3.1: The system shall provide an admin dashboard displaying KPIs: Total Users (Candidates/Employers), Active Job Listings, Application Rate, and Platform Growth Trends.
- 4.3.2: Admins shall have enhanced tools to manage user accounts (suspend, verify), moderate job postings/company profiles, and view system audit logs.

FR7.2: Message Management (Optional - Implemented)

- 7.2.1: Users shall be able to send and receive real-time text messages within established connections (e.g., post-application).
- 7.2.2: A user shall be able to delete any conversation thread from their own inbox.
- 7.2.3: A user shall be able to block another user, preventing further messages from the blocked party.
- 7.2.4: A user shall be able to report a conversation or specific messages for inappropriate content, flagging it for admin review.
- 7.2.5: Admins shall be able to view reported messages and take action (warn user, suspend account, delete content).

FR6.1: Notification System (Optional - Implemented)

- 6.1.1: The system shall deliver real-time notifications to users via WebSocket connection for the following events: Connection requests (received, accepted) , Job applications and status updates , Company follows and engagement , Post interactions (likes, comments)
- 6.1.2: The system shall provide a notification bell icon in the navigation bar that:
  - Displays a badge with the count of unread notifications
  - Opens a dropdown panel showing recent notifications when clicked
  - Updates in real-time without requiring page refresh
- 6.1.3: Users shall be able to:
  - View all notifications in chronological order (newest first)
  - Mark individual notifications as read/unread
  - Mark all notifications as read with a single action
  - Distinguish between read and unread notifications via visual indicators
- 6.1.4: Each notification shall contain:
  - Notification type (connection, application, follow, post interaction, system)
  - Timestamp of the event
  - Relevant message or description
  - Read/unread status

7. Post Feed System Requirements

7.1: Post Display and Feed

7.1.1: Main Feed Display

The system shall display a post feed that:

- Shows posts in a scrollable feed format
- Displays 7 posts per page by default
- Supports infinite scrolling or pagination
- Updates in real-time when new posts are created
- Provides visual loading indicators during content fetch

7.1.2: Personalized Feed

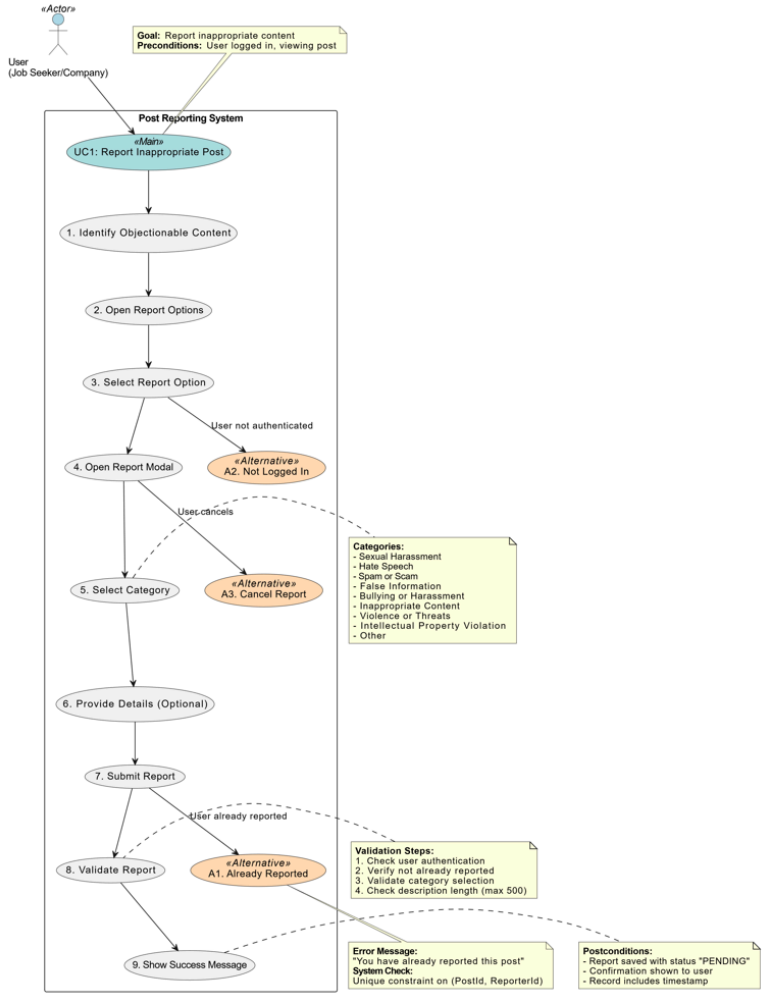
When a user is logged in, the system shall:

- Display a personalized feed based on user connections and interests
- Calculate post relevance scores using multiple factors
- Cache personalized feeds in Redis for improved performance
- Update feed rankings when new interactions occur

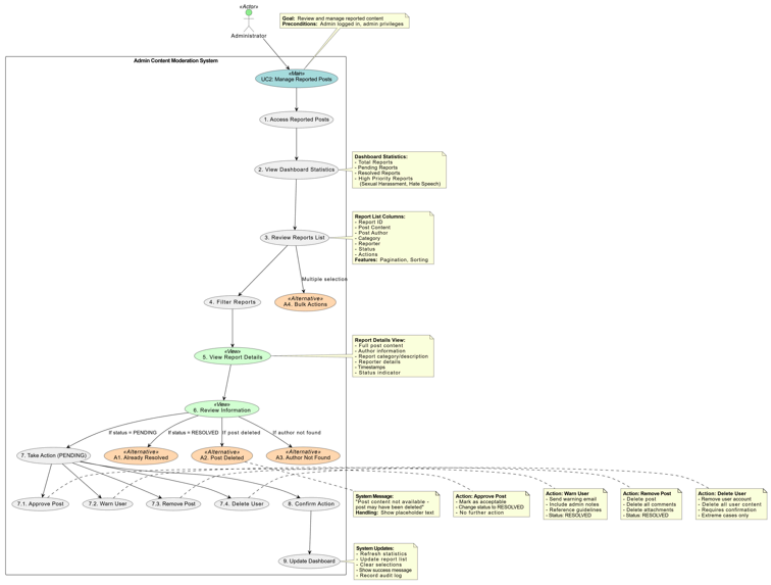
---

5. Important Use Case and Sequence/Activity diagram Summary

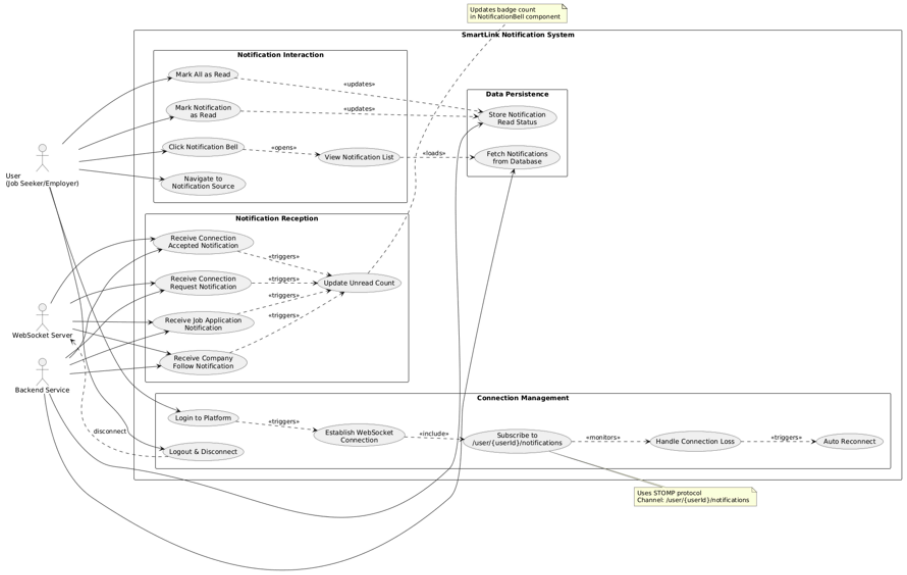
Use Case 1: User Report Post (PlantUML)



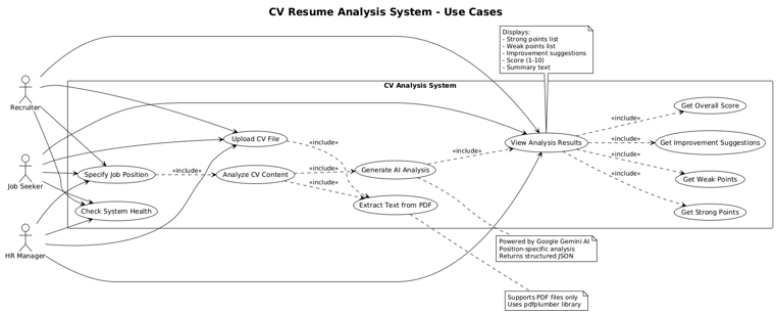
Use Case 2: Admin Management of Reported Posts (PlantUML)



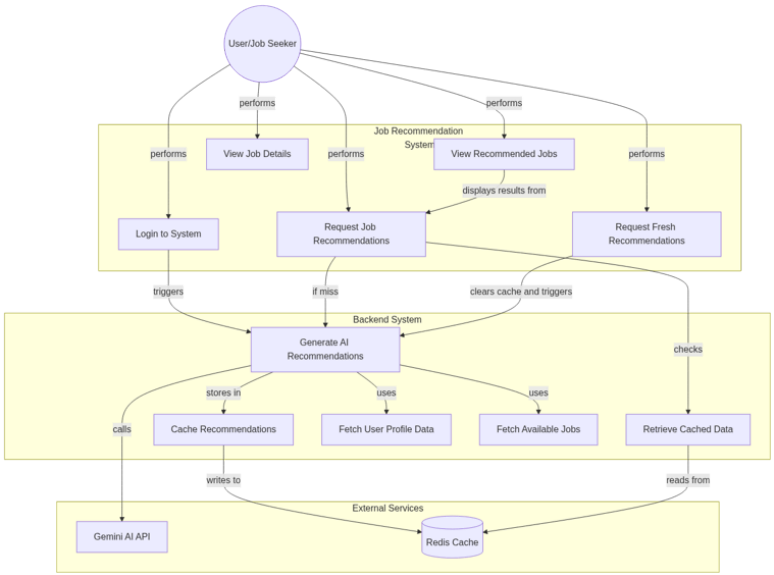
Use Case 3: Notification System (Plant UML)



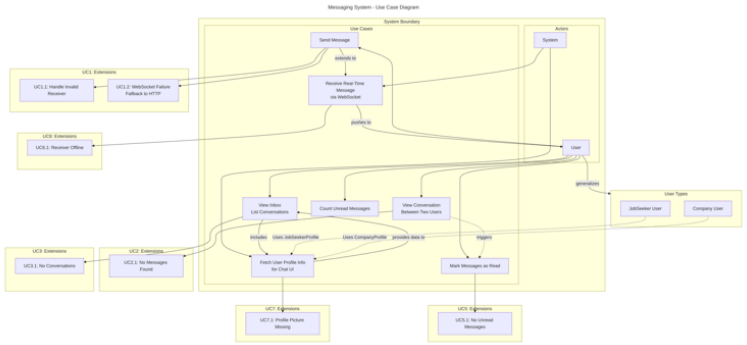
Use Case 4: Resume Analysis



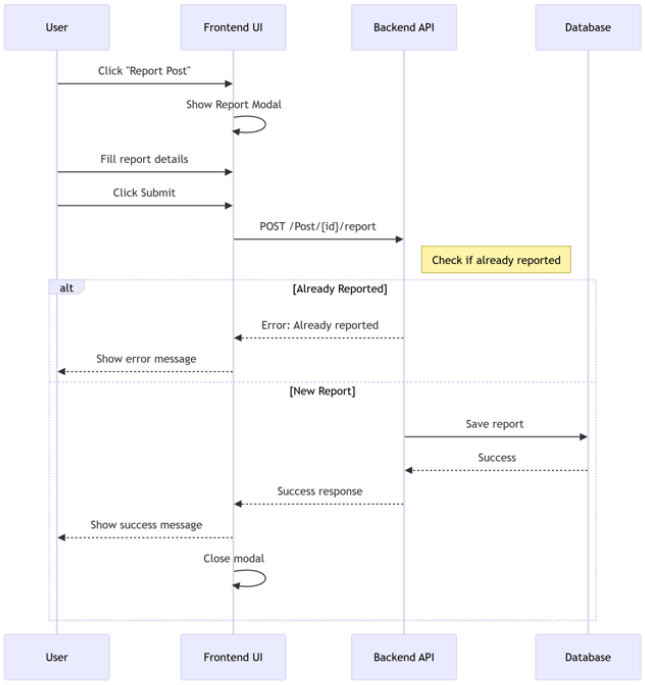
Use Case 5: Job Recommendation System



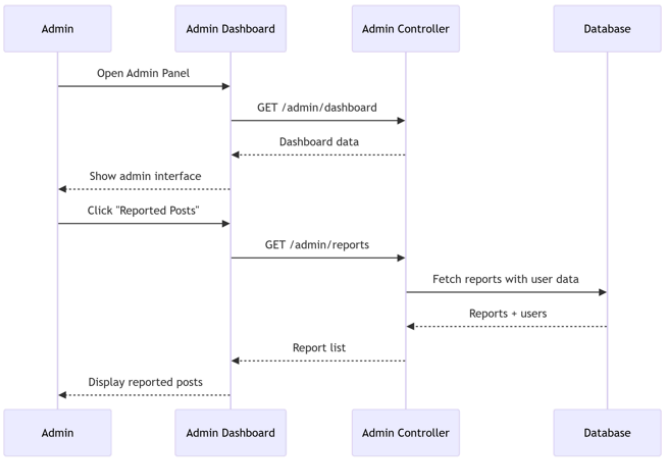
use case 6: Message System



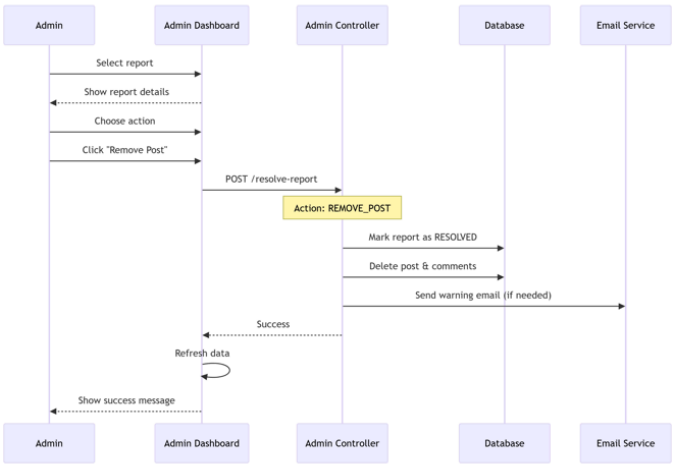
Sequence Diagram 1: Simplified User Report Post



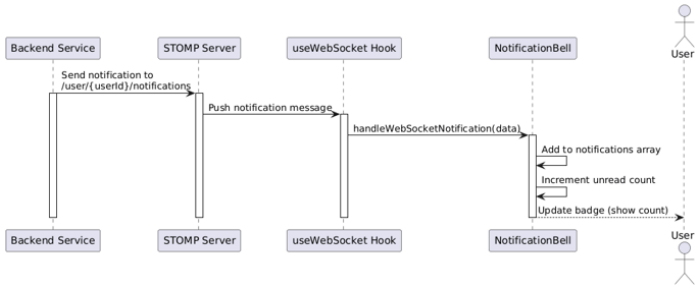
Sequence Diagram 2: Admin Dashboard Access



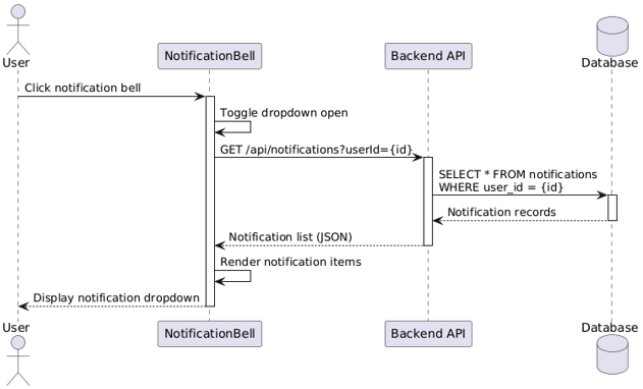
Sequence Diagram 3: Admin Take Action



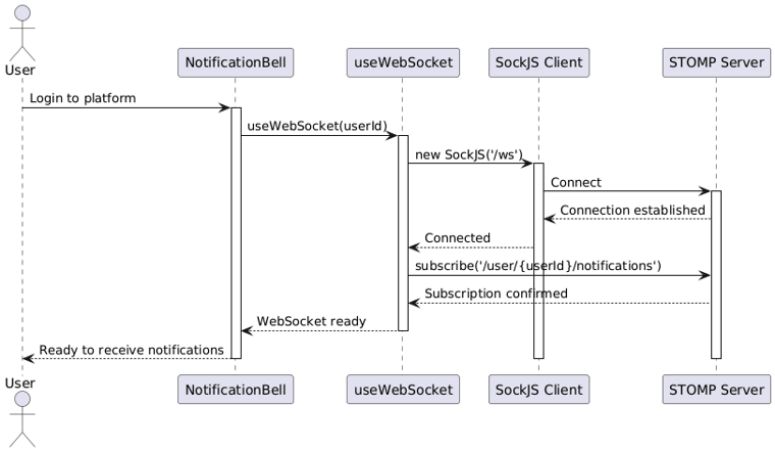
Sequence Diagram 4: Receiving Real-time Notification



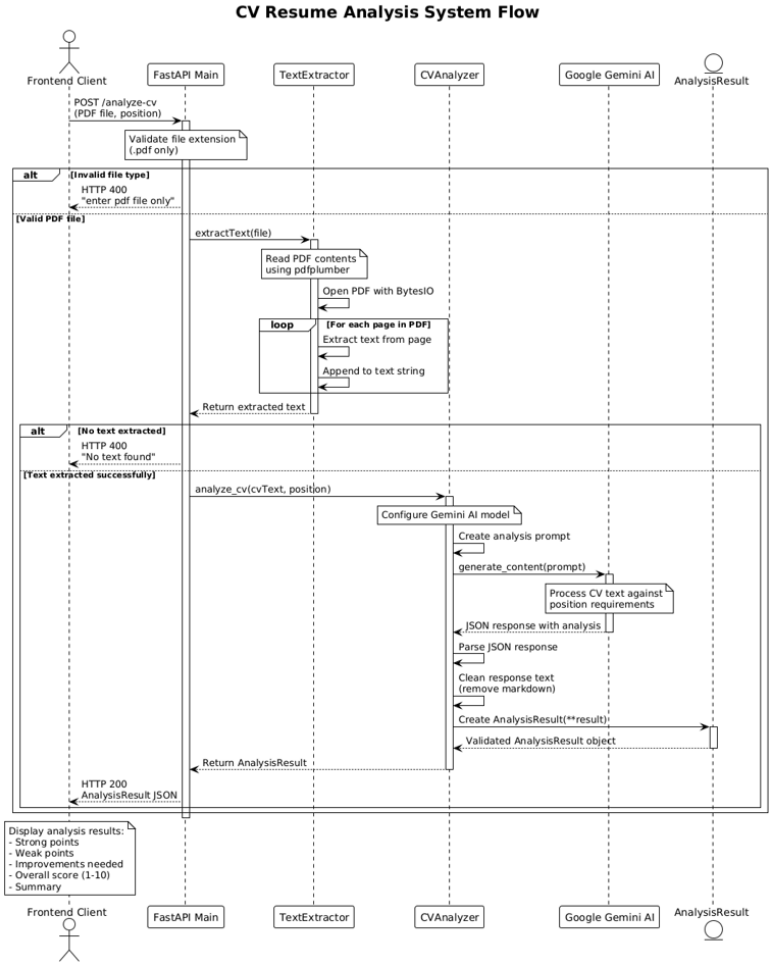
Sequence Diagram 5: Viewing notification list



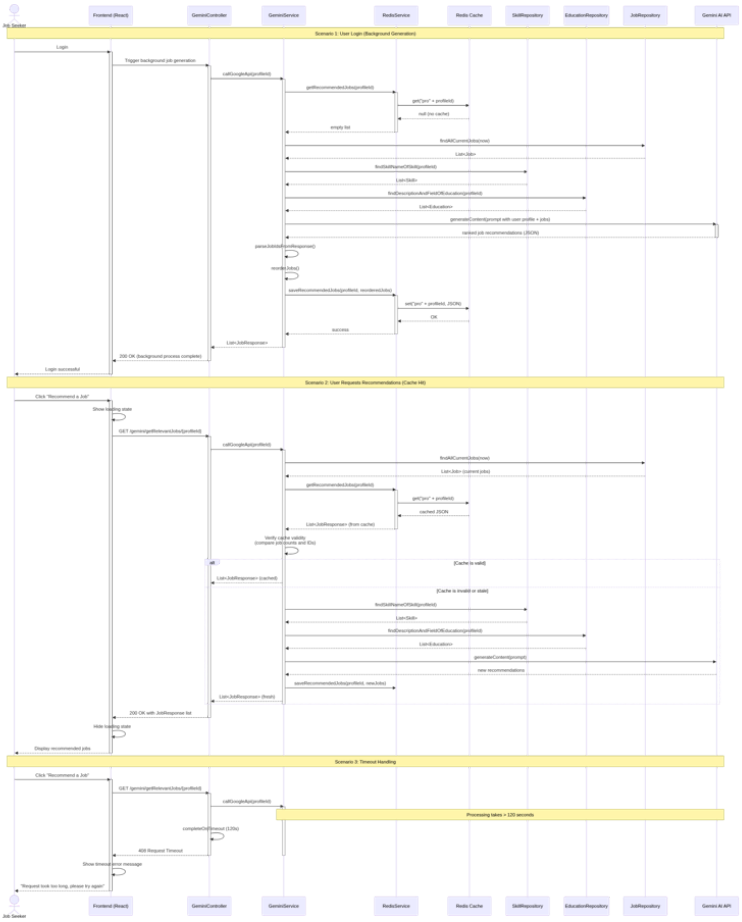
Sequence Diagram 6: WebSocket Connection Establishment



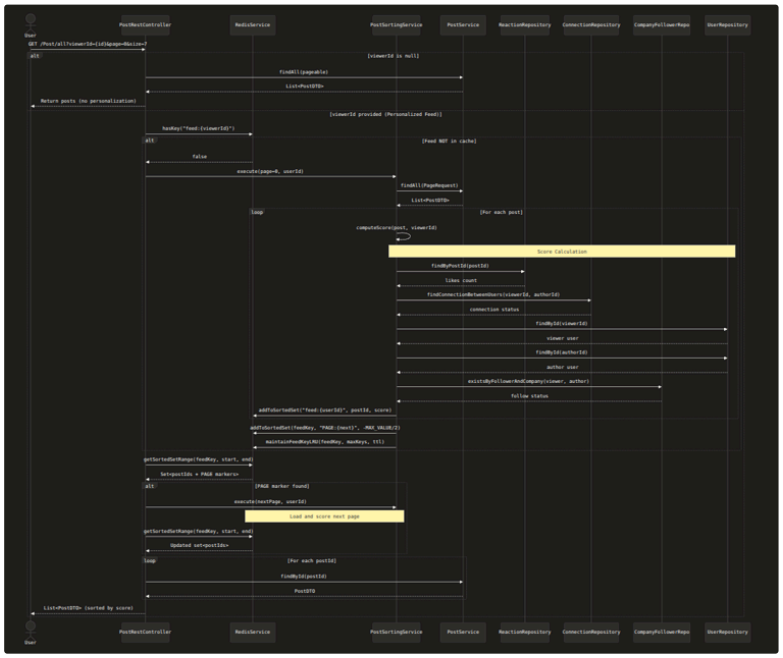
Sequence Diagram 7: Resume Analysis



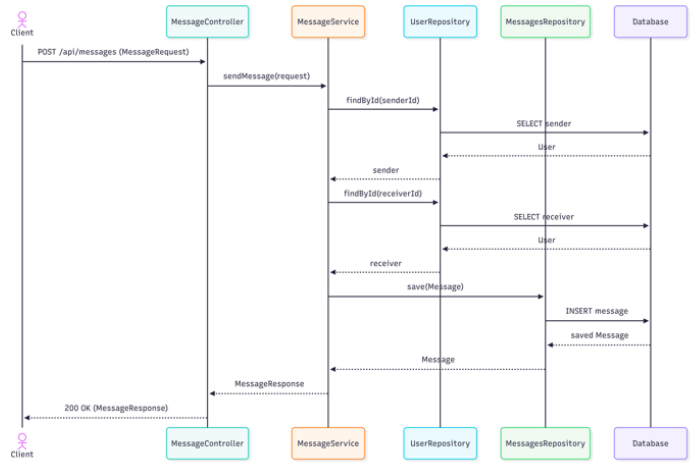
Sequence Diagram 8: Job Recommendation System

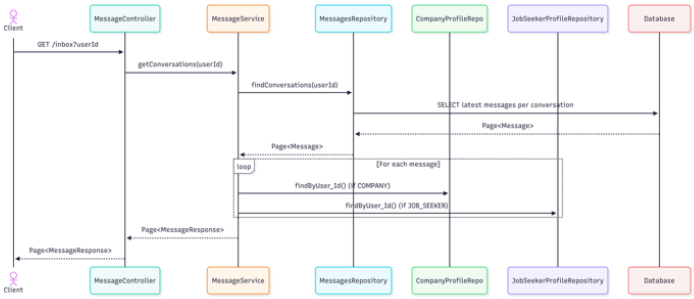


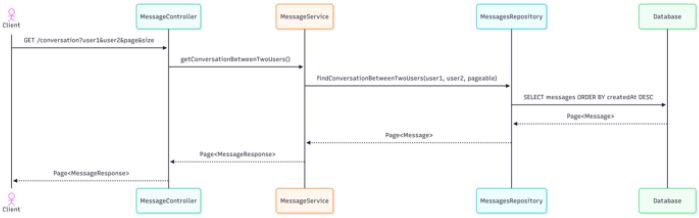
Sequence Diagram 9: Post Recommendation system



9. Sequence diagram 10: for live chat







6. System Architecture

Initial Design

- Monolithic structure
- Direct UI-to-database communication

Improved Design

- Layered Architecture:
  - Presentation Layer: React
  - Business Layer: Spring Boot Services
  - Data Layer: Repositories & MySQL & Redis

Benefits

- Better security
- Clear separation of concerns
- Improved scalability
- Easier maintenance

---

7. Workflow & Project Management

Development followed Agile methodology using Jira for task management.

---

8. Testing Strategy

Unit Testing

- Backend coverage: more than 60%
- Tools used: JUnit & Mockito
- TDD approach applied

---

9. Version Control & Collaboration

- GitHub used for source control [omar2004khaled/SmartLink: Social app with authentication and user management]
- Each member created and reviewed Pull Requests
- All PRs linked to Jira tasks for traceability [Smart-Link - Backlog - Jira]

Prepared by: SmartLink Development Team